

Invoice Audit: Write-up

Khaled Alamro, September 2026

How I measured my results

The only ground truth in the package is the set of labels for hospital 1, and that is also the data I used to build and debug the checker. A score on that set says more about how well I fitted it than about how the checker will behave on hospitals 2 to 5, so I treated it as an upper bound and looked for other ways to measure.

On hospital 1 the final pipeline flags all 58 erroneous invoices with no false alarms, matches every error category, and reproduces the expected total on 909 of the 913 invoices. To see how much of that came from tuning, I made each fix I had added after looking at the labels switchable. The very first run already found all 58 invoices without a false alarm. The later fixes corrected five category labels and one total, and turning them all off changes only 20 of the 3,942 rows I submitted. That told me the headline result was not simply a product of fitting.

For the scored hospitals, the risk I worried about most was reading a contract wrongly, because a misread rate silently affects every invoice that uses it. I checked the extracted rules in three independent ways. First, the parsers run 830 consistency checks of their own, such as confirming that "eighty" and "(80)" agree. Second, an LLM read each contract separately, and I compared its output field by field with the parser's. There were no differences, and I confirmed the comparison would have caught one by planting a one-cent change. Third, since most invoices are correct, a misread rate would show up as a service whose billed prices mostly disagree with my calculation. The lowest agreement for any service was 96%.

Because hospitals 2 to 5 have no labels, I also measured detection directly. I planted 2,040 known errors into invoices my checker considered clean, across all five hospitals and 22 error types. These included the amendment-date issue in hospital 3 and the multiplier issue in hospital 5. The checker flagged 99.2% of them, and every error of the standard and contract-specific types was flagged with the right category. The 16 misses all came from two types I deliberately made difficult, and none of them was passed as clean with high confidence: all 16 went to the human review queue.

Confidence is assigned in fixed tiers that I set before scoring and then checked against hospital 1, rather than fitting them to it. No tier turned out to be overconfident.

There are two things I could not measure. The stress test only contains errors of the kinds I thought to plant. And nothing in the package lets me measure how often the checker raises a false alarm on hospitals 2 to 5.

Where I was uncertain, and why

The clearest example is invoices that exceed a daily cap. The contract says the excess is not payable, so I expected those invoices to be paid at the cap. In all four such cases in hospital 1, however, the labels pay fewer units than the cap allows, which looks like the original quantity before the error was introduced. That quantity is not in the data. I kept the contract reading, kept a high confidence that these invoices are wrong, and marked the amount itself as uncertain. The 28 affected invoices are in the review queue.

The second was a clause about volume discounts, which counts the "units billed" for a service. It was not clear whether a line billed in the wrong unit should count. Leaving those lines out produced five findings sitting exactly on a discount threshold, which is what a wrong reading would look like. So I counted them and gave those five invoices a lower confidence.

Matching invoice descriptions to contracted services was uncertain by nature, because hospitals abbreviate and drop words. The hardest case is a service that is not in the contract but, with one word dropped, looks like one that is; in testing, about 1% of these slipped through unnoticed. I also found two limits in my own design. A line billed in the wrong unit cannot be priced, so a price error on the same line goes unseen, which affects 54 flagged invoices. And a broken date on one line can remove a bundle rate from another, making a correct price look wrong. I chose to document these with examples rather than patch them at the last minute.

Some clauses were never exercised by the data, such as hospital 2's Service Day starting at 07:00 when invoices only carry dates. I used the literal reading, confirmed that none of them changes an output, and recorded each one in the decision log.

Finally, the brief says two hospitals done well is better than four done thinly. I submitted all four, because the checking engine is shared and every contract passed all three extraction checks. I believe that was the right call, but it is a judgement rather than something I could prove.

What I would do differently with another week

I would build the error-planting test on the first day instead of near the end. It turned out to be the only real way to measure the unlabelled hospitals, and having it earlier would have exposed the blind spots while I was still designing.

I would also check a random sample of around a hundred invoices from hospitals 2 to 5 by hand. That would give me an actual false-alarm rate and let me set confidence from real data rather than from the set I built with.

In a real audit, I would ask the payer about the cap convention and the 07:00 Service Day early on, instead of choosing a reading and working around it. Until I had an answer, I would report both amounts.

With the remaining time, I would close the gaps in the design: price wrong-unit lines under each plausible unit, treat findings caused by another broken line as consequences rather than separate errors, and improve the matching for look-alike services.

I stopped at about 4.6 of the 6 to 8 hours, because the remaining items needed either answers I did not have or more time than the cap allowed. I used Claude Code as an assistant throughout, and the prompts are included in the repository.